

ICML 2023 | Lining Up a Graph So a CNN Can Read It

Compressed Convolution Networks (CoCN) learn a differentiable permutation that arranges every node in a row, bringing image-style convolution to graphs — with state-of-the-art node and graph classification.

Convolutional neural networks owe much of their success to two plain ideas: local filters that scan a regular grid, and weight sharing that reuses those filters everywhere. On images this yields multi-scale features almost for free. Graphs, unfortunately, are not grids. Nodes have no canonical order and neighborhoods vary in size, so the convolution that works so well on pixels has no obvious analogue.

The usual workaround is to build graph filters out of polynomials of the adjacency matrix. These convolutional GNNs are permutation invariant and elegant, but under a tight parameter budget they are not very expressive, and they cannot learn hierarchical, multi-scale representations on their own without bolt-on tricks like node clustering or node-drop pooling.

"All in a Row: Compressed Convolution Networks for Graphs" (ICML 2023) takes a different route. Instead of redesigning convolution for graphs, it reshapes the graph so that ordinary convolution applies. The model learns a task-specific, differentiable permutation that lines all of a graph's nodes up in a single row, then slides a CNN-style kernel along them. The resulting model, CoCN (Compressed Convolution Network), learns node and structure features end to end and reaches the state of the art on both node and graph classification.

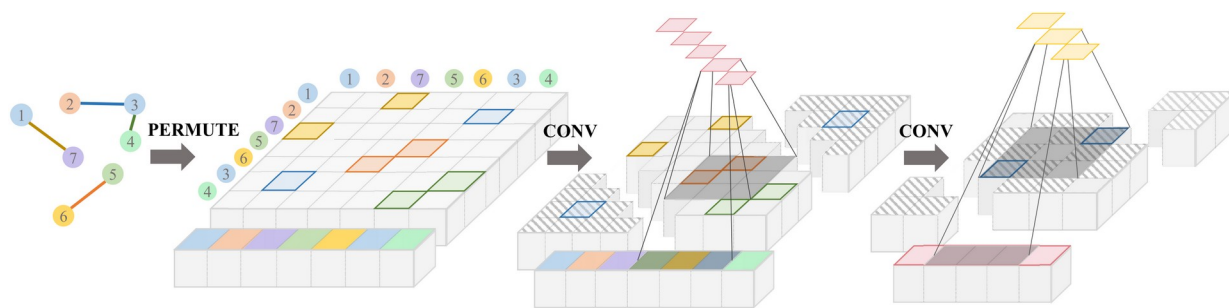


Figure 1. The CoCN pipeline. A learnable permutation matrix reorders the node-feature and edge-feature matrices so related nodes sit next to each other; a diagonal convolution then slides along the diagonal of the permuted matrices, extracting each node's features and the local structure around it together. Off-diagonal entries (twill pattern) are left for later layers.

Paper: <https://proceedings.mlr.press/v202/sun23k.html>
<https://github.com/sunjss/CoCN>

Code:

The tension: order-sensitive convolution, order-invariant graphs

Convolution is deliberately order-sensitive — move the kernel one step and you read a different patch. A graph neural network must be the opposite: relabel the nodes and the prediction should not change. Earlier attempts to reconcile the two chose a node ordering as a

preprocessing step, but that ordering was fixed independently of the downstream task, so it could not be tuned and often discarded useful structure. What CoCN adds is an ordering that is learned end to end and provably permutation invariant.

Learning a permutation instead of choosing one

CoCN has four modules — input, permutation generation, convolution, and output — and the heart of it is permutation generation. For each node it regresses a scalar position with an MLP on the node features, smoothed by the graph Laplacian so that connected or similar nodes receive nearby positions. Similar nodes end up as neighbors in the row, exactly the arrangement a local kernel wants.

Those raw positions are turned into a relaxed permutation matrix: a differentiable, soft stand-in for a hard reordering, controlled by a factor τ that provably sharpens into a true permutation as it grows. Because the ordering is learned by regression rather than searched over, the cost of permutation modeling drops from $O(n!)$, which is hopeless, to $O(n^2)$ — cheap enough to train in mini-batches on graphs with hundreds of thousands of nodes.

Diagonal convolution and compressed pooling

Once the nodes sit in a row, CoCN abandons the row-by-row, column-by-column sweep of image convolution for a diagonal one: the kernel slides along the diagonal of the permuted edge-feature and node-feature matrices, reading each node together with the local structure around it in a single pass. Compressed pooling then squeezes the structure matrix from the anti-diagonal direction to build a hierarchy of coarser graphs, and transposed convolution (TConv) upsamples the features back when the task needs a label for every node.

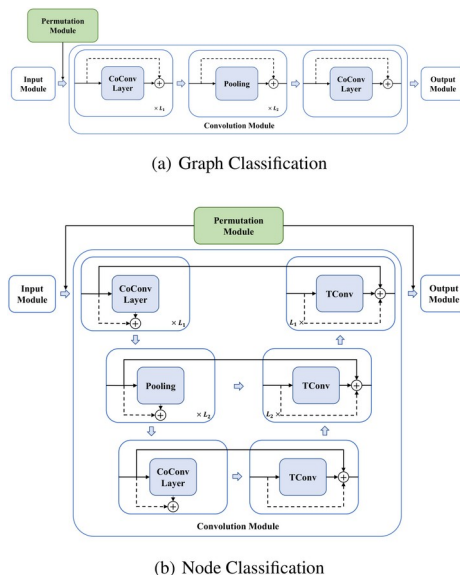


Figure 2. CoCN architectures for the two tasks. For graph classification (top), compressed convolution and pooling layers stack into an encoder. For node classification (bottom), transposed-convolution layers mirror the pooling stack to upsample features back to node resolution, U-Net style. The permutation module feeds both.

What the permutation actually learns

If the idea works, the learned ordering should place related nodes next to each other, turning a scrambled adjacency matrix into something with visible block or band structure that a local kernel can exploit. Figure 4 shows exactly this: as the Laplacian smoothing t increases, the permuted adjacency matrix and the node-feature similarity matrix concentrate their mass into blocks and diagonals rather than scattering it across the grid.

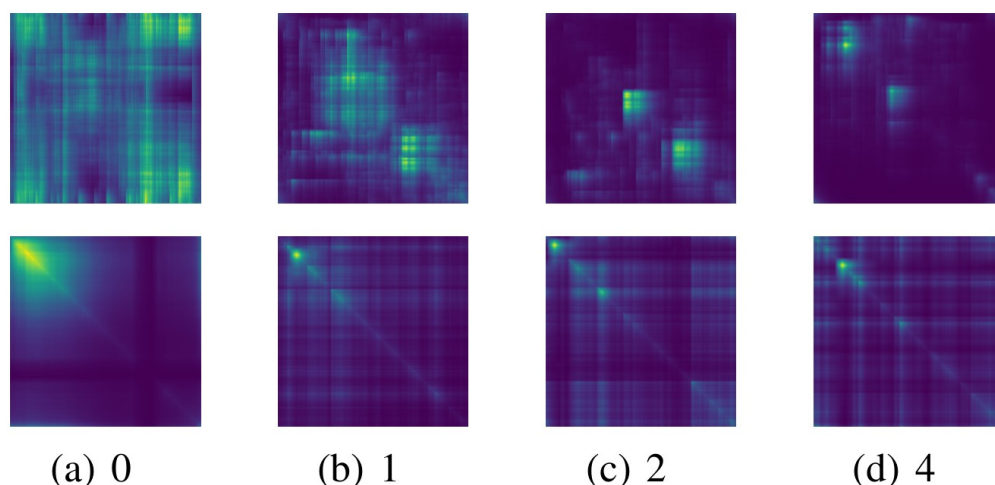


Figure 4. Permutation results at different smoothness t . Top row: the permuted adjacency matrix, where brighter pixels mark connected nodes. Bottom row: the permuted node-feature similarity. As t grows, block and diagonal structure sharpens — precisely the local pattern a diagonal kernel can compress.

The emerging block structure is what makes a small diagonal kernel effective: the information a node needs is now local, just as it is in an image patch.

A fair worry about any relaxation is that softening a hard permutation loses information. The authors test this directly with a graph-reconstruction experiment: an autoencoder is trained to recover the original features from the permuted ones, so the reconstruction error measures how much the relaxed permutation throws away.

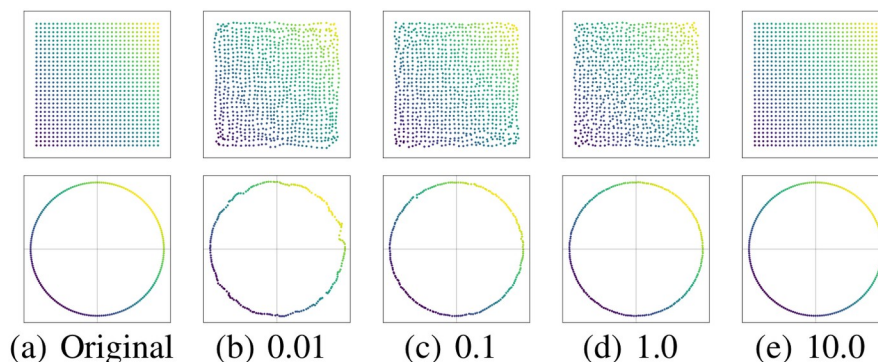


Figure 3. Reconstructing a grid (top) and a ring (bottom) from relaxed permutations at increasing relaxation factor τ . Even at a moderate τ the geometry is recovered almost exactly, indicating the differentiable permutation is nearly lossless.

Reconstruction is nearly perfect once τ is moderate, so the differentiable relaxation buys trainability at almost no cost in information — the soft permutation preserves essentially what a hard one would.

Results

The payoff shows up on hard benchmarks. On six heterophilic node-classification datasets, CoCN reaches the best average rank, 1.83, topping the field — including 79.17% on Chameleon, 72.95% on Squirrel, and 86.22% on Cornell. The Squirrel result is the headline: it beats the strongest prior model, LINKX, by more than eleven points (72.95% versus 61.81%). On graph classification it wins five of six TU datasets, with 86.15% on COLLAB and 56.32% on IMDB-MULTI.

Heterophilic benchmarks are known to contain duplicate nodes that can leak labels, so the authors also evaluate on filtered versions of Chameleon and Squirrel with those duplicates removed. CoCN still ranks first among 18 models, which suggests its advantage is not an artifact of leakage.

Table 1. Node-classification accuracy (%) on the original and filtered Chameleon and Squirrel datasets; the filtered splits remove duplicate nodes. CoCN ranks first on both original datasets and stays competitive after filtering.

Model	Chameleon Ori.	Chameleon Filt.	Squirrel Ori.	Squirrel Filt.
FAGCN	64.23	41.90	47.63	41.08
GCN	50.18	40.89	39.06	39.47
ResNet+adj	71.07	38.67	65.46	38.37
FSGNN	77.85	40.61	68.93	35.92
CoCN (Ours)	79.61	41.95	73.27	39.69

Why does lining nodes up once help, and would lining them up in several different ways help more? Each permutation exposes the convolution to a different arrangement, so CoCN learns several in parallel. Figure 5 shows accuracy rising steadily as the number of permutations grows on MUTAG, Chameleon, Cornell, and Wisconsin.

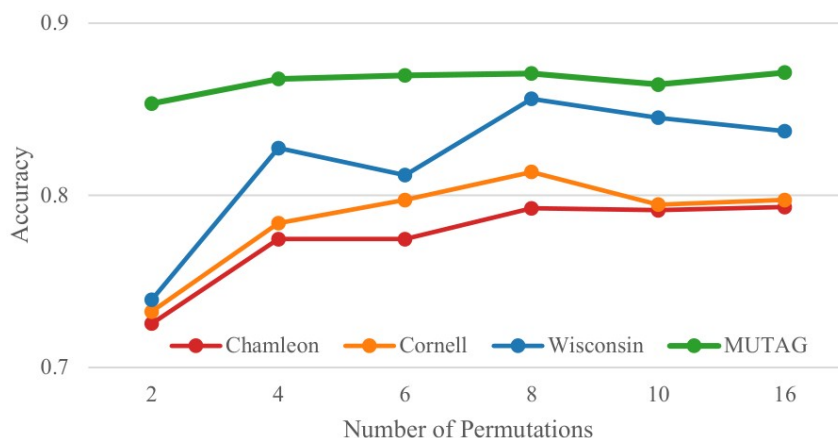


Figure 5. Ablation on the number of permutations. Accuracy climbs steadily as more permutations are added across four datasets, then flattens after roughly 8 — a handful of complementary orderings captures most of the benefit.

The gain flattens after about eight permutations, which is a practical knob: a small set of complementary orderings captures most of the benefit, and the model gradually learns to make those orderings less redundant.

Takeaway

CoCN's contribution is conceptual as much as empirical. If you can learn a differentiable permutation that arranges a graph's nodes in a row, the entire toolkit of image convolution — sliding kernels, pooling, hierarchical features, upsampling — becomes available for graphs, with an $O(n^2)$ cost that scales. The honest boundary: the authors note the gains are smaller on weakly connected graphs such as MUTAG and Texas, where there is less local structure for a diagonal kernel to compress. Where structure exists, though, treating a graph as an image you first have to sort turns out to be a strikingly effective idea.

When to reach for CoCN

As a practical rule of thumb, CoCN pays off when a graph has real local structure to sort — community or homophily patterns, or clear node-feature clusters — because that is exactly what the diagonal kernel compresses. On near-structureless graphs the gain shrinks, as the MUTAG and Texas results show. Two properties keep it worth a place in the toolbox regardless: the ordering is learned end to end rather than hand-designed, and its $O(n^2)$ cost lets a CNN-style model train by mini-batch on graphs with hundreds of thousands of nodes — a regime where most convolution-on-graph ideas simply do not scale.

Paper: <https://proceedings.mlr.press/v202/sun23k.html>
<https://github.com/sunjss/CoCN>

Code: